scientific reports

OPEN

Comparison of metaheuristic algorithms set-point tracking-based weight optimization for model predictive control

Kawsar Nassereddine✉ & Marek Turzynski

Traditional controllers frequently perform poorly when managing complex industrial systems because they must balance multiple goals like cost, pollution, and efficiency. For higher performance and efficiency, model predictive controller addresses this but necessitates efficient cost function tuning, which is currently frequently enhanced utilising metaheuristic algorithms. This work aims to develop and validate a data-driven weight optimisation method for multivariable model predictive controller in a DC microgrid comprising photovoltaic panels, battery, supercapacitor, grid, and load—by systematically balancing control effort and accuracy. Four algorithms—particle swarm optimization, genetic algorithm, pareto search, and pattern search-selected for their capacity to resolve complex, multi-objective problems are used in this study's automated, set-point tracking-based weight optimisation approach. The results show that incorporating parameter interdependency reduces genetic algorithm's power load tracking error from 16% to 8%, while particle swarm optimization achieves an error of under 2% even without considering interdependency. Fast convergence and trade-offs are supported by pareto and pattern search, but they are less responsive to sudden changes. These methods provide a feasible, repeatable way to boost model predictive controller performance.

Keywords Model predictive control, Weight tuning, Particle swarm optimization, Set-Point tracking, Sensitivity analysis

Managing complex systems in industrial processes like power generation, chemical manufacturing, and oil refining requires addressing the issue of optimisation under several, sometimes conflicting goals, like lowering emissions without sacrificing product quality or minimising energy consumption while increasing efficiency. To discover a fair trade-off between objectives including cost, efficiency, safety, and environmental effect, these situations call for multi-objective optimisation strategies. For instance, speeding up production in chemical plants may result in increased energy consumption and safety hazards; in power generating, maximizing output might interfere with fuel economy. To solve this problem, a Model Predictive Controller (MPC) can be a useful tool for managing these trade-offs.

MPCs are advanced management strategies that forecast future behaviour and identify the best course of action by using a model of the system. MPC facilitates proactive decision-making that improves overall system performance by carefully handling competing goals. Although MPC is good at handling complicated systems, adjusting the cost function is one of its biggest problems. The cost function's ability to effectively weight different goals is crucial to MPC efficiency, and striking the correct balance frequently necessitates thorough research and might take a lot of effort. This tuning problem is still very important and requires careful and effective techniques to solving it[1].

Numerous metaheuristic optimisation techniques have been employed in recent research to optimise or adjust a MPC's cost function. Techniques like Genetic Algorithms (GA), Particle Swarm Optimisation (PSO), Salp Swarm Optimisation, and Differential Evolution systematically investigate the weighting-matrix space, surpassing manual trial-and-error in closed-loop tasks ranging from industrial process control to arterial blood pressure regulation. Due to their ability to balance computing efficiency and search width, comparative surveys show that PSO and GA are the most used techniques; nevertheless, newer swarm-based approaches are being used more for complex, nonlinear plants[2].

Faculty of Electrical and Control Engineering, Gdansk University of Technology, Gdansk, Poland. ✉email: nassereddinejana@gmail.com

The problem of effectively MPC tuning is not new, and some approaches have been proposed and described in the literature. For example, in[3] a PSO-based tuning approach for MPC under model uncertainty was discussed. Solution was validated on the Shell heavy oil fractionator process and optimized for handling multiple control objectives. This approach is flexible, robust, and can be applied to various MPC formulations. Another PSO-based method for tuning networked MPC systems in large-scale networks like electricity grids and enabling a balance between performance and communication overhead was described in[4].

In[5], GA and multi-objective fuzzy decision-making method was introduced to optimize MPC tuning parameters based on user-defined control preferences, and it was validated through a mixing tank case study. The technique demonstrated its flexibility, adapting well to different preference sets. Similarly, in cement kiln control, GA-based MPC tuning was used to minimize tracking error and energy consumption, with an interactive decision tree involving operator input for real-time adjustments. This operator-driven approach was more effective and faster than multi-objective techniques, outperforming them in a variety of contexts by 23.74% in various settings[6].

Another interesting solution was described in[7], where a multi-objective optimization method was applied to adjust MPC in a Crude Distillation Unit, optimizing weight matrices either sequentially or simultaneously to balance output tracking and input movement, yielding lower computational costs compared to traditional methods. Likewise, two multi-objective optimization techniques were presented for MPC tuning: lexicographic optimization for square systems and compromise optimization for non-square systems, demonstrating reduced commissioning time and less computational effort. These methods were successfully applied to the Shell Heavy Oil Fractionator subsystem[8]. A machine learning architecture was introduced for automating the tuning of many inputs and many outputs MPC in a diesel engine air path, optimizing tuning time and cost by approximating the cost function[9]. Additionally, an adaptive MPC tuning method was proposed, using online identification and Linear Matrix Inequalities with Recursive Least Squares for parameter updates. This method is effective for managing unknown or time-varying parameters and is more efficient than conventional techniques[10].

In[11] an application of Bayesian Optimization for tuning MPC controllers in Heating, Ventilation, and Air Conditioning systems was proposed. This approach enables elimination of the need for lengthy closed-loop simulations, what results in better performance, lower computational cost compared to traditional methods and offers higher efficiency for MPC applications. Described solution was adapted and tested to complex building control.

Similarly, Bayesian Optimization may be also employed for autonomous vehicle lateral control, adjusting to various driving scenarios and successfully completing lane-keeping maneuvers in simulations. This tuning technique optimized MPC gains based on user-specified standards[12].

A multi-objective optimization procedure for Nonlinear MPC to balance reference tracking and control effort was proposed in[13]. This approach was validated on the Van de Vusse reactor and Tennessee Eastman plant, demonstrating robustness to disturbances and model mismatch. Another method for NMPC systematically determines tuning parameters through multi-objective optimization, ensuring strong closed-loop performance despite disruptions. This technique was applied to three chemical process examples described in[14].

In[15], an analytical tuning technique for multivariable MPC was proposed for square First-Order Plus Dead Time systems with inactive constraints, focusing on decoupling and achieving the intended performance through closed-loop transfer functions. Additionally, a technique for MPC tuning in autonomous quadrotors used machine learning to adjust objective function weights, compensating for nonlinearities and modifying the system to avoid obstacles during flight, demonstrating the potential of machine learning for MPC in Aerial vehicule operations[16]; Although it requires a large amount of training data and adds additional processing cost, this data-driven approach learns and adjusts dynamically to complicated nonlinear dynamics.

Another approach was presented in[17], where a simplified method for tuning MPC by dynamically adjusting output setpoints rather than internal weights was described. This technique offers desired control performance with minimal computational effort and supports zone control and real-time optimization.

An MPC tuning is used to build thermal control[18], achieving up to 5.2% energy savings and improved comfort by adjusting prediction and control horizons, outperforming conventional PID control; however, the approach demands higher computational resources and accurate system models, which can limit deployment in legacy building-management systems.

And to automate MPC tuning, reinforcement learning (RL) may be also used. For example, in[19] this approach was adapted to control aerial robot trajectory tracking. The proposed solution, validated through real-world experiments, significantly reduced the time and effort typically required for weight tuning in MPC. Nevertheless, RL's high computational cost and training phase instability are significant drawbacks, particularly when used in safety-critical systems.

In[20] An offline technique for adjusting MPC parameters in limited multi-inputs multi-outputs systems was presented. This approach enables to reach a balance between the accuracy and response speed while ensuring closed-loop stability. Using exponential data weighting allowed long time horizons prediction without sacrificing computational efficiency. Achieved results demonstrated its superiority over existing techniques in terms of both performance and speed. However, the approach's primary drawbacks are limited capacity to adjust to changes in real time and the requirement for precise prior system modelling.

Another solution was discussed in[21], where a robust closed-loop performance tuning method for MPC was proposed. Described approach was validated through a steering control case study and focused on resilience in the face of disturbances. To ensure robustness and anticipation in control design matrix-based methods were used. In another approach, robustness-oriented tuning for AutoRegressive with eXogenous input model (ARX) based offset-free MPC was demonstrated for multivariable systems such as cement mills and distillation columns[22]. Nevertheless, this approach's primary drawback is its dependence on precise ARX model identification.

In a review of MPC, tuning techniques was presented. Authors discussed a specialized methods like Max-Plus-Linear MPC and general approaches at the importance of accurate process models for effective tuning. A usefulness of Kalman filtering, state estimation, and auto-tuning were also discussed. Authors also noticed that advancements in subspace identification and autocovariance least-squares had made MPC tuning more cost-effective[23].

In[24] a unique technique for improving MPC weights in motion cueing algorithms for driving simulators was proposed. This approach utilizes a multi-objective genetic algorithm to optimize the cost function weights for objectives such as minimizing motion inputs and improving realism. The technique was further refined using an interactive genetic algorithm (IGA) to maximize user satisfaction. Simulation results indicated that the method enhanced the realism of driving simulators and reduced motion inaccuracies compared to previous tuning techniques. However, it has limited scalability to real-time or large-scale applications, relies on subjective user feedback for IGA, and has a high computational cost.

Similarly, a strategy for tuning MPC in automated and connected vehicles (CAVs) in mixed traffic was described in[25]. Bayesian optimization and inverse reinforcement learning were applied to adjust weights online, achieving a 20% improvement in performance at unsignalized crossings, showcasing its effectiveness in real-world traffic scenarios. However, this strategy necessitates sufficient information availability, which reduces its efficacy in situations with limited or extremely variable data.

A novel tuning strategy for nonsquare systems with more outputs than inputs was introduced in[26]. Proposed method is based on analysis of set points or ranges for controlled variables. The methodology includes determining the best attainable closed-loop performance, calculating optimal scaling, and tuning the controller weights. The method was successfully applied to the Shell heavy oil fractionator benchmark, demonstrating steady performance and robust disturbance handling without excessive modification of manipulated variables.

To optimize the trade-off between fatigue loads and power generation Pareto curves may be effectively used to assess a model predictive control (MPC) strategy for wind turbines. By using this technique, MPC weights can be adjusted to lower loads without compromising power production[27,28]. As it is presented in[29], pattern search methods for linearly constrained problems can provide global convergence to Karush-Kuhn-Tucker points without the need of gradients. This might be further simplified when the constraints are restricted to variable bounds.

Recent research demonstrates a clear trend toward hybrid and intelligent optimization methods for modern energy systems. Hybrid schemes such as GOA-PSO–based PID controllers have achieved substantial improvements in settling time and overshoot reduction in renewable-rich scenarios[30], while Adaptive MPC frameworks enhanced with reinforcement learning and optimizers like COA-jDE have improved frequency stability and grid code compliance in low-inertia systems[31]. Similarly, approaches such as MGOA-tuned PD-PIDA controllers and GOA-PSO–tuned STATCOMs have been shown to strengthen fast frequency response, LVRT compliance, and post-fault stability in microgrids[32,33]. In the PV domain, fuzzy logic–based MPPT strategies, including modified and hybrid variants, have enhanced tracking accuracy and speed, though at the cost of added complexity[34], while comparative analyses of FLC and MPC highlight FLC's superior dynamic response against MPC's long-term stability[35]. Together, these works reflect the field's growing emphasis on sophisticated, hybrid, and adaptive optimizers to address the nonlinear and highly variable challenges of renewable-dominated power networks, providing essential context for our algorithmic comparison.

An example of the system that requires multi-criteria optimization is also a microgrid power system. In suc h systems (e.g. DC microgrids), energy is generated by various distributed power sources (wind generators, PV, etc.) - Fig. 1, whose level of power generation depends on many factors. Mutual correlation of these factors (e.g. irradiance, wind speed) is often difficult to predict in wider time horizon, what complicates system stabilization. As a result, in microgrid power systems use of additional energy storages (batteries, super capacitors etc.) is required. Hence, a precise prediction of energy storages state of charge depending on consumers' energy consumption and weather conditions plays an important role in system optimization and stabilization. On the other hand, microgrid power systems usually cooperates with system power grids, thus the bidirectional energy flow occurs between these systems. It must be noted that energy cost is not constant, and it varies in dependency

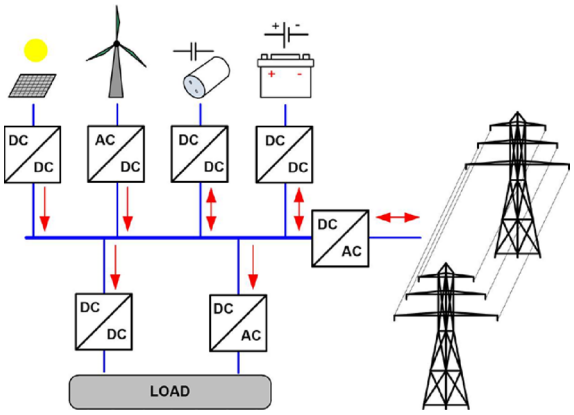

Fig. 1. Concept of DC micro grid power system.

on the time of day and year season. Consequently, an economic aspect must be also considered in a microgrid power system control process. To solve microgrid optimization problem, an approach based on application of MPC may be effectively adopted[36].

This work's primary goal is to create and verify a weight optimisation technique for multivariable MPC based on set-point tracking. This entails developing a performance-driven loss function that accounts for control effort and tracking accuracy, putting in place a systematic tuning procedure with simulation feedback, and testing the strategy on representative control systems as shown in Fig. 2. The objective is to offer a repeatable, data-driven framework that improves control quality under various operating conditions and accelerates MPC weight selection.

The structure of this article is as follows: The controller setup is explained in the "MPC Setup Model" section, which also covers the derivation of the cost and loss functions. The four optimisation techniques utilised to adjust the cost function weights for the model predictive controller are presented in Section "Optimisation Algorithms". The sensitivity analysis of important parameters for the optimisation algorithm is shown first in the "Results" section. This is followed by a comparison of the load tracking performance using the tuned MPC with different optimization strategies. Finally, the "conclusion" offers a summary of the main findings and future work.

MPC setup model

A hybrid energy storage system consisting of a battery and supercapacitor is managed using a linear MPC designed using MATLAB software, which regulates power flow from the grid, load, and battery while taking solar PV disturbances into consideration. The controller functions use is a state-space model[37], with measured disturbance[38] from PV panels. For inputs and states, both hard and soft constraints are used[37]. Equation 1 penalises deviations in State of Charge (SoC) and power tracking using a custom cost function J, it involves 20 weights to be optimized:

$$\begin{aligned} J = & \sum_{k=1}^{N_p} \left[(S1\sigma + (1-\sigma)\, S2)\, (SOC_{bat}(t+k) - SOC_{bat,ref})^2 \right] + \sum_{k=1}^{N_p} \left[(S3\sigma + (1-\sigma)\, S4)\, (SOC_{SC}(t+k) - SOC_{SC,ref})^2 \right] \\ & + \sum_{k=1}^{N_p} \left[(S5\sigma + (1-\sigma)\, S6)\, P_t(t+k) * P_{grid}^2(t+k) \right] + \sum_{k=1}^{N_p} \left[(S7\sigma + (1-\sigma)\, S8)\, (P_{load}(t+k) - P_{dem}(t+k))^2 \right] \\ & + \sum_{k=1}^{N_u} (\alpha\, 1\sigma + (1-\sigma)\, \alpha\, 2) P_{batt}^2(t+k) + \sum_{k=1}^{N_u} (\lambda\, 1\sigma + (1-\sigma)\, \lambda\, 2)\, \Delta\, P_{batt}^2(t+k) \\ & + \sum_{k=1}^{N_u} (\alpha\, 3\sigma + (1-\sigma)\, \alpha\, 4) P_{grid}^2(t+k) + \sum_{k=1}^{N_u} (\lambda\, 3\sigma + (1-\sigma)\, \lambda\, 4) \Delta P_{grid}^2(t+k) \\ & + \sum_{k=1}^{N_u} (\alpha\, 5\sigma + (1-\sigma)\, \alpha\, 6) P_{Load}^2(t+k) + \sum_{k=1}^{N_u} (\lambda\, 5\sigma + (1-\sigma)\, \lambda\, 6) \Delta\, P_{Load}^2(t+k) \end{aligned} \quad (1)$$

where S_i, α_i, and λ_i are the weights for each variable, N_p is the prediction horizon and N_c is the control horizon; $SOC_{batt}(k)$, $SOC_{SC}(k)$ are the state of charge of battery and supercapacitor, $P_{grid}(k)$, $P_{batt}(k)$ and $P_{loads}(k)$

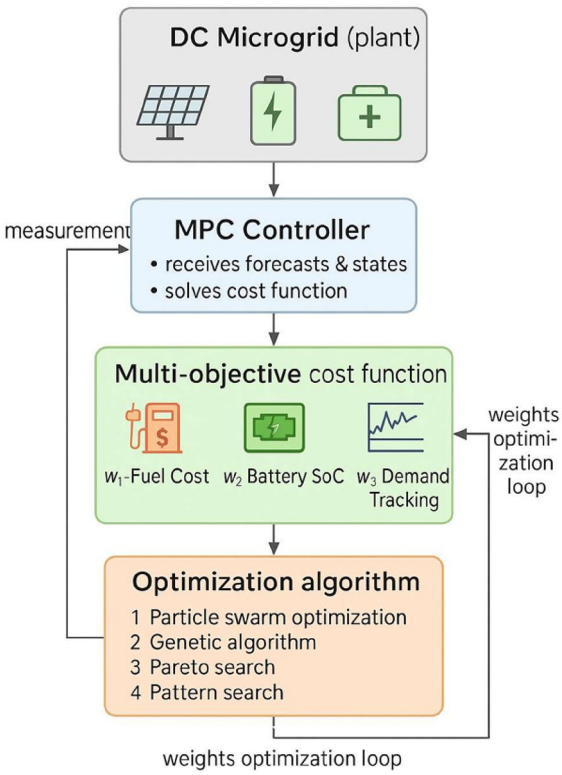

Fig. 2. MPC-based DC microgrid control with weight tuning via multi-objective optimization.

represent the grid, battery and load power respectively, while $P_{dem}(k)$ representing the demand power, σ is a Binary variable indicating curtailment period and finally $P_f(k)$ is the fuel price;

The metaheuristic optimisation algorithms (PSO, GA, Pareto, and Pattern Search) evaluate and choose the best weights for the custom cost function, as illustrated in Fig. 3.

A main script imports system parameters, then chooses one of four optimisation techniques (PSO, GA, Pareto, or Pattern Search), where the loss function evaluates performance each time by running the MPC using the current weight vector. A consistent framework that guarantees accurate comparison of control outcomes is used throughout this iterative process, which is carried out in MATLAB.

Custom cost function

Key components of DC microgrid operation, including precise power tracking, smooth energy flow, and battery life preservation, are optimised using the cost function in this work. It includes terms that reduce variations from the battery's and supercapacitor's reference state of charge, hence extending storage life. Other terms encourage system stability and energy efficiency by penalising big or sudden variations in power flow as well as deviations between the desired and actual load power.

This cost function (1), is based on MATLAB's default MPC structure[37], and has been altered to incorporate demand monitoring error and grid power use penalties, making it more appropriate for actual microgrid scenarios with intermittent renewable energy sources. A binary switching variable adjusts the weightings during low-generation periods—such as early morning or late afternoon—to prioritize energy saving and reliability.

Loss function

The loss function evaluates the performance of a given set of weights by running the MPC with the current parameters, constraints, initial conditions, and system data. The main component of the loss is the tracking error's squared norm, and to avoid overfitting, a regularisation term penalises high weight values. The final loss is the sum of the tracking error and the regularization term, with the regularization scaled by eta = 5 to appropriately balance its impact relative to the tracking error. All these elements are shown in (4), (5) and (6) respectively. Further clarification of the parameters is given in (2) and (3).

$$\mathrm{P_{iw}} = \left[\mathrm{X_{mpc}}\,(1,\ :)'\,;\ \mathrm{u_{mpc}}\,(3,\ :)'\right]; \tag{2}$$

$$\mathrm{R_{Bi}} = \left[\mathrm{data.socBatRef};\ \mathrm{data.P_{dem}}\right]; \tag{3}$$

$$\mathrm{Ew} = (\mathrm{norm}(\mathrm{P_{iw}} - \mathrm{R_{Bi}}))^2; \tag{4}$$

$$\mathrm{R_w} = (\mathrm{norm}(\mathrm{W}))^2; \tag{5}$$

$$\mathrm{L} = \mathrm{E_w} + \mathrm{eta} * \mathrm{R_w}; \tag{6}$$

where.

- $X_{mpc}(1, :)$: Predicted battery state of charge over the prediction horizon.
- $u_{mpc}(3, :)$: Predicted grid power.
- *data.socBatRef(1: end)*: Reference trajectory for battery state of charge.
- $data.P_{dem}(1: end\text{-}1)$: Forecasted power demand.
- *W*: vector representing penalized quantities such as control effort or slack variables.
- *L*: The total cost function combining tracking error and regularization penalty.
- *eta*; A weighting factor that scales the influence of the regularization term in the total cost.

The Optimization Algorithms uses a loss function to tune the optimal weights in the MPC cost function, with the goal of minimizing the tracking error between load and demand[39]. It also guarantees that the storage system's state of charge (SoC) stays within predefined boundaries[37].

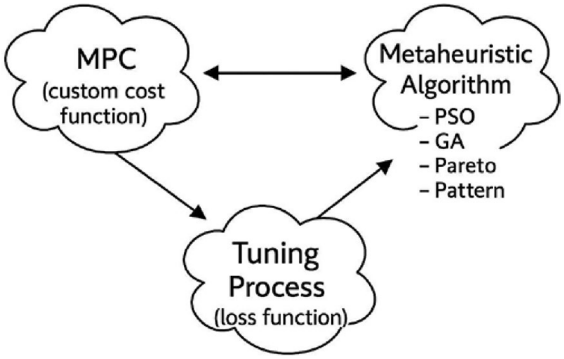

Fig. 3. Metaheuristic-Based Cost function tuning framework for MPC in DC Microgrids.

Optimization algorithms

This work uses different types of optimisation methods illustrate in Fig. 4 to automatically adjust the weights of the cost function used in the MPC. Comparing diverse approaches—population-based, multi-objective, and local search— these methods provide a comprehensive evaluation of their strengths in tuning MPC weights for improved load tracking accuracy. And to maintain system stability and effective power dispatch, the primary goal is to reduce the tracking error between the actual demand and the predicted load power. By simulating the MPC with candidate weight sets and measuring how well the provided power matches the load requirement, the loss function is used to assess controller performance and direct optimisation towards the optimal weighting settings.

Particle Swarm Optimization (PSO)

Every potential weight vector is represented by PSO as a "particle" in a multidimensional search space. These particles travel across space by adjusting their position and velocity in response to the swarm's global best solution as well as their own best past performance. Without the need for gradient information or an explicit system model, this collective learning behaviour aids in determining the optimal weight configuration. The search space research and the exploitation of suitable regions are encouraged by this hybrid movement. Using random components helps prevent local minima and guarantees diversity in the search. The swarm gradually approaches the area of the ideal or nearly ideal solution. For each particle *i*, the velocity and position are updated as shown in (7) and (8)[40]:

$$\mathrm{v_i}\,(\mathrm{t}+1) \;=\; \omega \cdot \mathrm{v_i}\,(\mathrm{t}) \;+\; \mathrm{C1} \cdot \phi 1 \cdot (\mathrm{pbest_i} - \mathrm{x_i}\,(\mathrm{t})) \;+\; \mathrm{C2} \cdot \phi 2 \cdot (\mathrm{gbest} - \mathrm{x_i}\,(\mathrm{t})) \tag{7}$$

$$\mathrm{x_i}\,(\mathrm{t}+1) = \mathrm{x_i}\,(\mathrm{t}) \;+\; \mathrm{v_i}\,(\mathrm{t}+1) \tag{8}$$

where:

- $x_i(t)$ and $v_i(t)$ denote the position (i.e., weight vector) and the velocity vector of particle *i* at time *t*;
- $pbest_i$ is the personal best position of particle *i*;
- *gbest* is the global best position among all particles.
- ω is the inertia weight (balances exploration and exploitation);
- *C1*, *C2* are the acceleration coefficients for cognitive and social learning; and.
- $\phi 1$, $\phi 2$ are random values between [0,1].

Each particle's position and velocity are randomly produced within predetermined bounds during the initialisation phase of the PSO method. The objective function is used to determine each particle's initial cost, and the best-known position is then determined for each particle. Subsequently, the particle having the lowest cost among all is determined to be the global best answer. The cost function, the number of particles, the number of dimensions (i.e., the weights being adjusted), the maximum number of iterations, and the algorithm-specific parameters—inertia weight and acceleration constants—are key inputs for the algorithm.

Each particle modifies its position and velocity according to three guiding terms during the main loop. The inertia term keeps the particle moving in the same direction (exploration), the cognitive component draws it in the direction of its most well-known position (exploitation), and the social component draws it in the direction of the swarm's global best solution. Particles are reassessed after updating and compared to boundary limitations. The particle's own best is updated if a new position results in a lower cost. In the same way, the personal best of any particle is updated if it surpasses the global best. PSO is particularly useful for adjusting complicated cost functions in control applications such as DC microgrids because of its capacity to strike a balance between exploration and exploitation. The specific parameter values used in this study were determined through sensitivity analysis and are listed in Table 1.

Genetic algorithm

Inspired by natural selection, a genetic algorithm is a population-based metaheuristic optimisation technique that finds optimal or nearly optimal solutions by evolving a population of candidate solutions, also known as chromosomes or individuals, over many generations; The algorithm assesses each individual's fitness at each

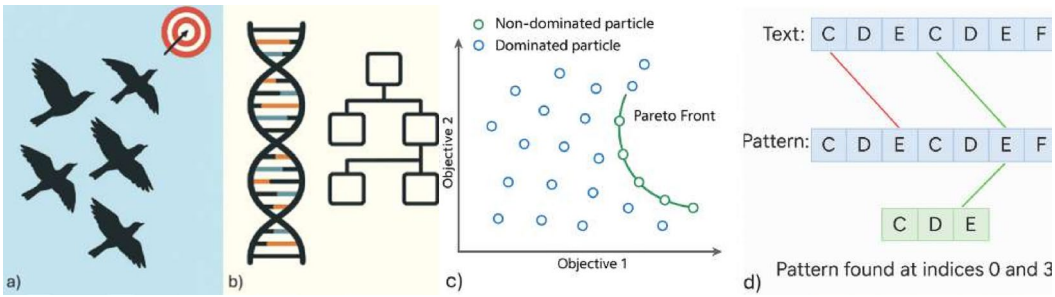

Fig. 4. Illustration of the four metaheuristics methods used: **(a)** PSO, **(b)** GA, **(c)** Pareto and **(d)** Pattern search.

Parameters	Values
Swarm size	2
Max Iterations	200
Lower and upper bounds	[0 100]
Inertia Range (w)	[0.5 1.2]
Social Adjustment weight	2
Self-Adjustment weight	1.501

Table 1. PSO algorithm parameters Settings.

Parameters	Values
Population size	10
Max Generations	50
Lower and upper bounds	[0 20]
Selection Function	Tournament
Mutation Function	Power
Crossover Function	Scattered
Elite Count	1
Crossover Fraction	0.6

Table 2. GA algorithm parameters Settings.

iteration (generation), chooses the most fit individuals for reproduction, applies crossover (recombination of parent solutions) and mutation (random alterations) to create new offspring, and then replaces some or all of the old population with the new one[41]. Additionally, Table 2 provide the suitable parameters for this method.

A population of ten individuals is first created at random by the genetic algorithm, which then iteratively evolves them through crossover, mutation, elite preservation, and selection. Each generation's fitness is assessed, and the most fit individuals are chosen for reproduction. This procedure is repeated until convergence, or a predetermined number of generations is reached. Equation (9) describes this process quantitatively, while the main loop is also provided down below.

For each Population, N individuals (chromosomes) are created within bounds as follows[42]:

$$X\ (0) = \ \{x1\ \ (0)\,, x2\ \ (0) \ldots, xN\,(0)\} \tag{9}$$

The primary loop of the GA starts with a fitness function evaluation of each individual. Using a tournament-style selection process, the most fit members of randomly selected subgroups are selected to become parents. A scattered crossover approach uses a crossover probability to make children by randomly combining genes from both parents. Then, with a specific probability, a mutation is performed, changing a gene in the descendants by appending a value selected from a power distribution. Elite preservation guarantees that the top individual or individuals (E = 1) are passed on directly to the following generation. Then, the newly produced descendants and elites are used to replace the existing individuals, updating the population.

MATLAB's default settings were used to establish the initial population for the Genetic Algorithm. These settings randomly create individuals between predetermined lower and higher bounds for each cost function weight. In addition to ensuring adequate coverage of the search field, this approach offers a wide range of potential options to begin the optimisation process. It is common practice to use the default initialisation, particularly in situations where no prior knowledge favours certain beginning locations[43].

Pareto search

When no objective can be improved without making at least one other target worse, the Pareto front is a set of nondominated alternatives. This is discovered using a multi-objective optimisation method called the Pareto search. When tackling issues with conflicting goals, this approach—which determines optimal trade-offs rather than a single ideal solution—is essential. By helping decision-makers understand and visualise the trade-offs between objectives, Pareto search practically enables them to make more informed and comprehensive choices in difficult circumstances. The parameters chosen are presented in Table 3. Equation (10)[44] provides a mathematical description of this process.

The technique creates poll points in the neighbourhood by appending step-scaled directions to each current candidates $x_k(j)$ at iteration k:

$$\mathrm{N_k\,(j) = \{x_k\,(j) + \Delta kp_i | i = 1,\ 2, \ldots, M\}} \tag{10}$$

where.

– Δk = step size at iteration k.

Parameters	Values
Maximum Function Evaluations	1000
Minimum Poll Fraction	1
Pareto Set Size	2
Lower and Upper Bounds	[0 100]

Table 3. Pareto algorithm parameters Settings.

Parameters	Values
Max Iterations	100
Initial Mesh Size	3
Max Function Evaluations	100

Table 4. Pattern algorithm parameters Settings.

– $p_i \in R_n$ = predefined polling directions (e.g., coordinate unit vectors).

Pattern search

Pattern search is a derivative-free optimisation method that progressively explores the search space by evaluating the objective function at several points organised in a pattern around the current answer. Pattern search is particularly useful for optimising functions that are noisy, discontinuous, or non-differentiable since, in contrast to gradient-based algorithms, it does not require any knowledge of the derivatives of the objective function. The algorithm repeats the procedure until a stopping criterion is met, iteratively expanding or contracting its search pattern to identify better solutions based on the results.

At iteration t, generate candidate points Ct around current solution Xt by adding or subtracting the mesh size Δt along coordinate directions; This procedure is mathematically described by (11)[45] and the parameters employed are listed in Table 4.

$$\mathrm{Ct} = \{ \mathrm{Xt} + \Delta \mathrm{te_i},\ \mathrm{Xt} - \Delta \mathrm{te_i} | \mathrm{i} = 1, \ldots, \mathrm{n} \} \tag{11}$$

where.

– e_i = unit vector along coordinate i,
– Δt = mesh size (step length) at iteration t.

Results

Sensitivity analysis of optimization parameters

The reliability of these algorithms-based tuning technique is further evaluated by a sensitivity analysis that concentrates on key algorithm parameters. By systematically changing these settings, the study discovers combinations that ensure accurate tracking outcomes, with performance assessed using the tracking error between the load power P_{load} and the demand reference P_{dem}, defined as Error% = $(P_{dem} - P_{load})/P_{dem}$. This phase is crucial to prevent overfitting and ensure that the controller functions dependably in a range of operating scenarios.

Particle swarm optimization

Effect of initial weight values on tracking performance Richards and Ventura[46] state that the particles should be initialized to ensure broad coverage of the search space, so that they are distributed as evenly as possible throughout the space. The standard method of initialization fails to accomplish this goal, especially in high-dimensional spaces.

We began our work using the default settings, adopting MATLAB's default parameters for PSO. The only modification was reducing the swarm size from 100 to 2 to significantly reduce computation time. After this, we ran the initial simulation. As shown in the top Fig. 5, the early stage results initially deviated significantly from the reference demand. To address this, we introduced initial values for the weights.

MATLAB's default settings, which produce individuals at random between predetermined lower and higher bounds for each cost function weight, were used to establish the initial population for the Genetic Algorithm. This approach helps guarantee enough coverage of the search field and offers a varied collection of potential options to begin the optimisation. When no prior knowledge favours certain starting positions, it is usual practice to use the default initialisation. The outcome, illustrated in the bottom figure, shows improved tracking performance, with values more closely aligned with the reference.

Maximum iterations The second parameter analysed in this sensitivity study is the maximum number of iterations, which defines the optimization depth in MATLAB's PSO. As noted in[47], few iterations can cause premature

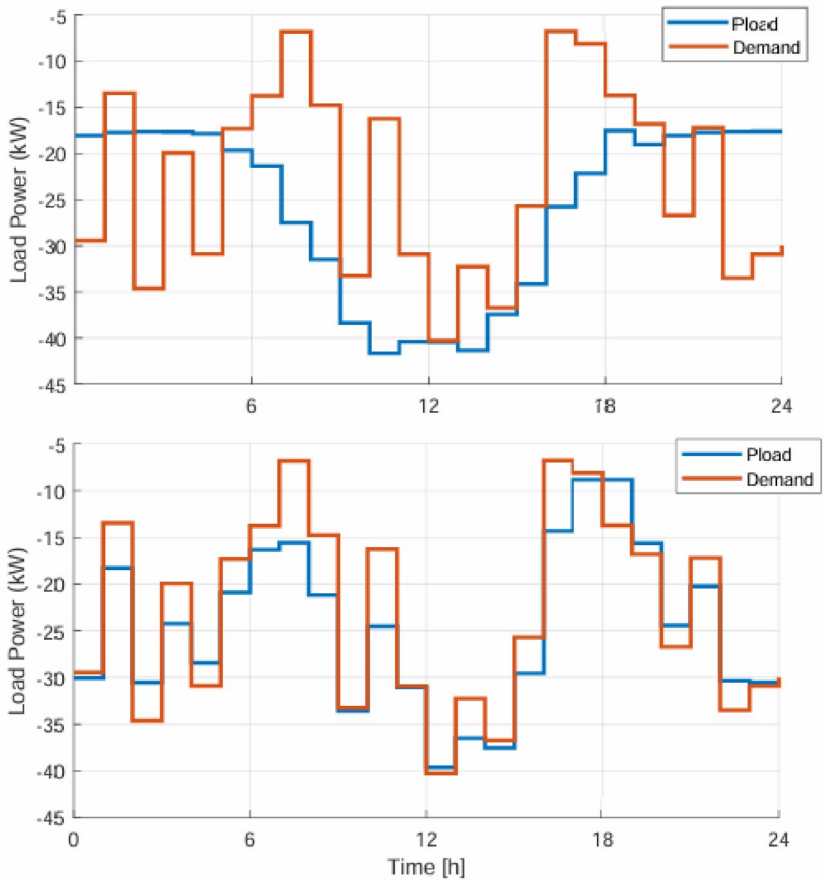

Fig. 5. Comparison of Output Tracking: **(a)** Default vs. **(b)** Initialized Weights.

convergence and limited exploration, whereas increasing the number of iterations improves the swarm's ability to reach the global optimum.

As shown in Fig. 6, similar results close to the reference can be obtained for values between 200 and 1000, indicating that 200 iterations are sufficient. However, when the maximum iteration is reduced to 100, the tracking performance noticeably deteriorates.

Initial swarm span The next parameter examined is Initial Swarm Span, which determines the initial range of particle positions in MATLAB's PSO. Particles cluster tightly when the starting span is too small, which reduces search diversity and leads to insufficient optimisation outcomes[48].

As illustrated in Fig. 7, values between 100 and 1000 yield results that closely follow the reference. However, when the span is set below 100, the tracking performance significantly declines.

Inertia range, social adjustment, self adjustment We begin by varying the inertia range (to explore how different levels of inertia affect the system's performance and to identify which range helps balance exploration and exploitation effectively). The inertia weight controls the impact of the previous velocity on the current velocity, influencing how much the system explores new solutions versus exploiting known good ones. Choosing an appropriate range is crucial because too high inertia can cause the system to overshoot optimal solutions, while too low inertia might lead to premature convergence[49]. After that, we identify the range that yields the smallest error.

Next, we fine-tune the social adjustment weights. Social weights determine how much influence the best solutions found by the group have on individual adjustments. By tuning these weights, we control how strongly particles (or agents) are attracted towards the global best position. As illustrated in Fig. 8(a), the error does not consistently increase with higher social weight values, so we select the value corresponding to the lowest error. This observation suggests that increasing social weight doesn't always lead to better or worse performance in a straightforward way; instead, there's an optimal value that minimizes error[50,51].

Finally, using the previously determined inertia and social weight values, we adjust the self-adjustment weight. Adjusting the cognitive weight after fixing inertia and social weights can fine-tune the balance between individual exploration and social learning, further minimizing error[52]. The self-adjustment weight controls the particle's tendency to rely on its own past best position.

In Fig. 8(b), we examined parameter interdependencies specifically, how the optimal value of one parameter (inertia, social, or self) depends on other parameter, and to confirm whether considering their combined effects

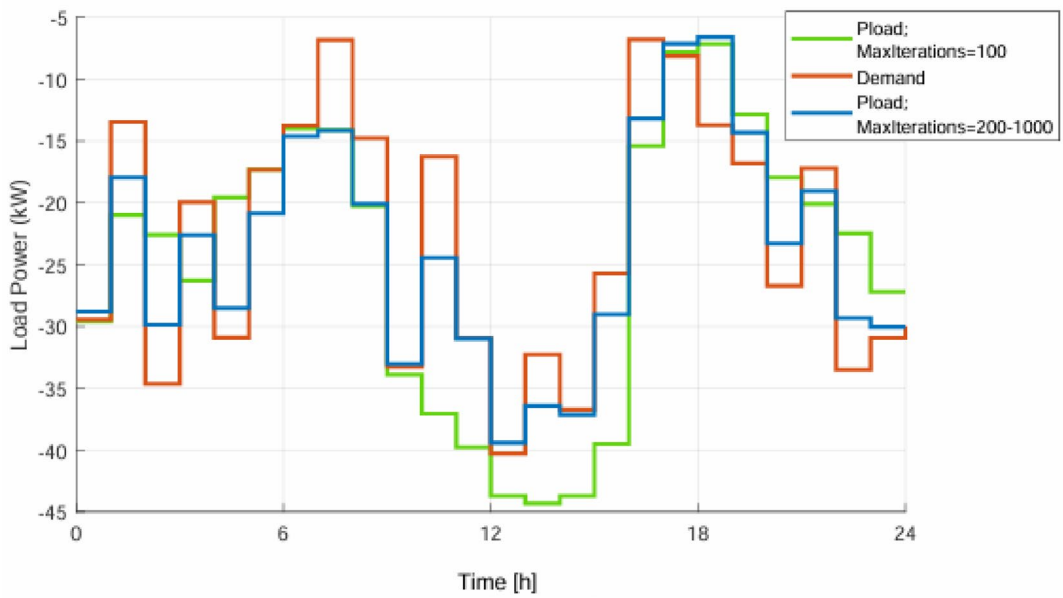

Fig. 6. Comparison of Tracking Performance for PSO Iterations: 100 vs. 200–1000.

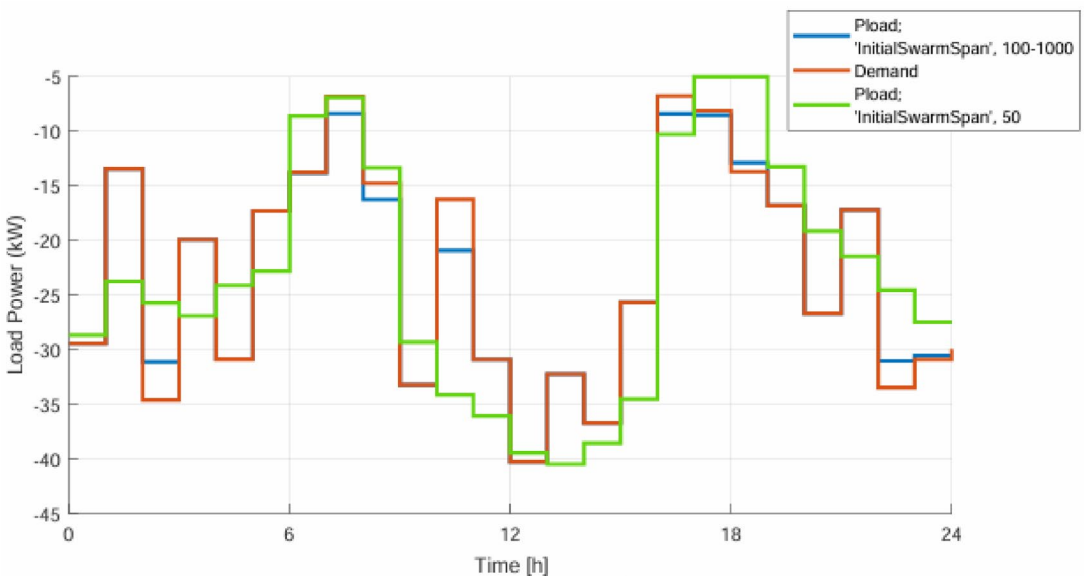

Fig. 7. Tracking Performance for Initial Swarm Span Values Below 100 and Between 100–1000.

could lead to improved optimization results. To capture these interactions, we used nested loops to evaluate the effect of each value of one parameter on the others.

Although no new minimum error below 1.96% was found, the analysis was essential to confirm that PSO parameters do not exhibit hidden interdependencies that could otherwise lead to unexplored improvements. Without this verification, one might wrongly assume that parameter tuning is fully independent, risking suboptimal settings or overfitting to specific scenarios. By confirming the absence of significant interdependencies, the analysis strengthens confidence in the reported optimal solution, demonstrating that the tuning reflects a thorough and reliable search rather than an incomplete or biased outcome.

Swarm size As the swarm size increases from 2 to the default value of 100, the simulation time also increases. However, the error does not follow a consistent trend: it initially rises from a swarm size of 2 to 5, then decreases between 5 and 20, and increases again beyond 20. Therefore, swarm size = 2 was chosen to minimize computation time, as it provided acceptable results without significant performance gains from larger swarm sizes as shown in Table 5.

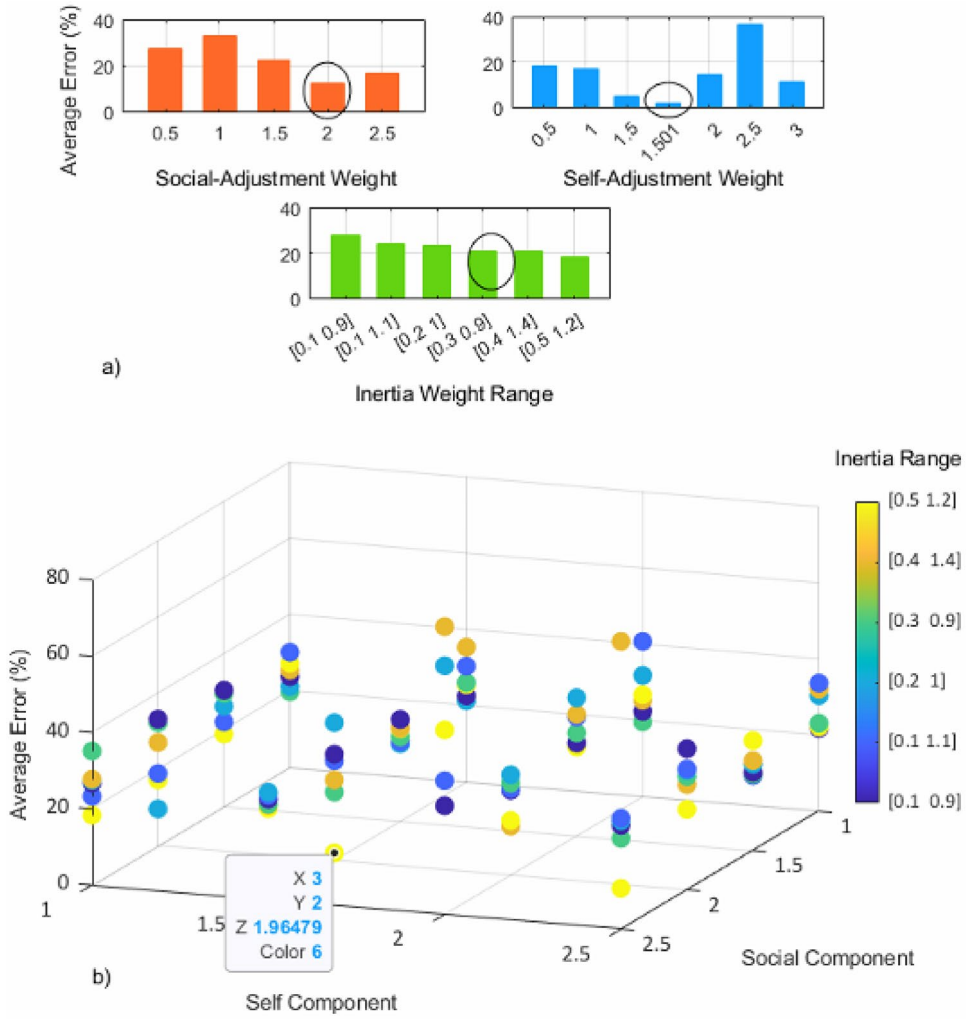

Fig. 8. Effect of PSO Parameters (c1, c2, and Inertia Range) on Average Error (%); **(a)** without interdependency, **(b)** with interdependency.

Swarm Size	Average Error %	Average time in minutes
2	1.96	2.5
5	8.87	6.2
10	4.57	12.5
20	4.27	52.7
30	14.8	127.2

Table 5. Effect of swarm size on simulation time and Error.

Optimal swarm size is problem-dependent; very large swarms do not always guarantee better performance and can sometimes degrade it[53], while larger swarms generally improve exploration and reduce the risk of local optimal[52].

Discussion Key PSO factors that impact load tracking was investigated in this sensitivity analysis, including swarm size, initial weights, iterations, swarm span, inertia, and social and self-adjustment weights. While 200 iterations and a swarm span over 100 offered a decent mix between accuracy and variety, setting starting weights greatly enhanced performance. Due to their non-linear impacts, optimal inertia and social weights needed to be carefully adjusted. Robustness was confirmed by parameter interactions, which revealed no superior combinations than sequential tuning. And finally larger swarms increased computation time, but a small swarm size of 2 offered acceptable results.

Genetic algorithm

Upper bounds The genetic algorithm's capacity to efficiently explore the solution space is strongly impacted by the selection of upper and lower constraints on the decision variables. While loose boundaries raise the complexity of the search space and may hinder convergence, tight bounds may limit exploration and result in premature convergence. In GA, establishing these limits appropriately is essential to striking a balance between exploration and exploitation[54].

When the upper bound is 30 or higher, tracking performance is acceptable during the curtailment period but poor outside of it. In contrast, with an upper bound of 20, tracking is better during non-curtailment periods but worsens during curtailment. An upper bound of 20 was chosen to maintain balanced tracking performance across all periods. While tracking during curtailment is not optimal at this setting, it can be improved by tuning other parameters.

Population size The error is significantly high when the population size is set to 2. As the population size increases from 2 to 10, the error decreases, reaching 16%. However, beyond a population size of 10, the error starts to rise again as shown in Fig. 9. Therefore, a population size of 10 was chosen as the optimal point[55]. Increasing the size further than 20 is not feasible, as it would significantly extend the runtime at the cost of other critical performance factors.

Other parameters In contrast to Particle Swarm Optimization, the Genetic Algorithm requires interdependency among parameters such as the selection function, crossover function, and mutation function to achieve lower error. Selection chooses individuals for reproduction based on their fitness; crossover combines them to generate offspring, while mutation introduces random variations to preserve diversity. Tournament selection offered a more balanced distribution, power mutation enabled efficient exploration, and scattered crossover helped reduce the error effectively[56]. This contrasts with PSO, which primarily depends on velocity and position updates without such operator interdependencies.

Before applying this interdependency according to the Fig. 10, the lowest error achieved was 15%. However, with the introduction of interdependent parameters, the error was reduced to 13%. This interdependency was further enhanced by adjusting additional parameters such as Elite Count and Crossover Fraction. Accounting for their interaction led to a new minimum error of around 8% in Fig. 11, a notable improvement over the 16% error.

Discussion The boundaries placed on choice variables have a significant impact on the speed of the genetic algorithm; tight restrictions run the risk of premature convergence, while loose bounds complicate searches and prevent convergence. To balance tracking accuracy during restriction and normal periods, an upper bound of 20 was selected. Error is also affected by population size, with 10 particles offering the optimal balance between computational cost and exploration. The interaction of selection, crossover, and mutation operators determines GA's efficacy in contrast to PSO; integrating these interdependencies decreased error from 15% to 13%, and fine-tuning elite count and crossover percentage further decreased it to roughly 8%. This emphasises how crucial it is to optimise GA parameters collaboratively to attain better outcomes.

By striking a balance between exploration and variety, GA parameters such as selection, crossover, and mutation can be tuned simultaneously to greatly improve performance. PSO parameters, on the other hand, behave more autonomously and are not as sensitive to these kinds of interactions.

Load demand tracking performance using optimization algorithms

PSO provided the most dependable load power tracking out of all the studied algorithms, retaining high accuracy both during curtailment and non-curtailment times. GA performed moderately well; under typical circumstances, its tracking was adequate, but during more dynamic phases, it lacked precision, which made it less appropriate for crucial energy management. Due to its trade-off balancing between various aims, Pareto optimisation performed almost as well as PSO during non-curtailment periods but became much less successful during curtailment. Like GA, pattern search produced average outcomes but was unable to adjust to the drastic

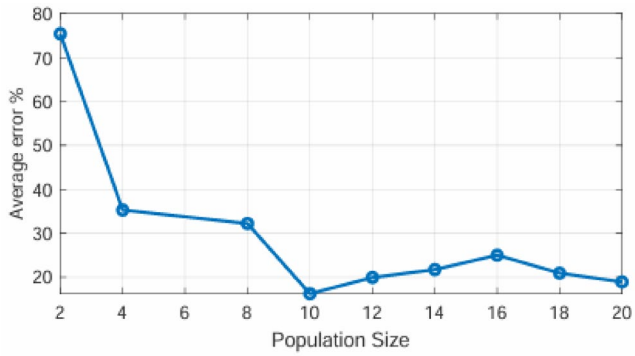

Fig. 9. Effect of Population Size on Tracking Error in GA Optimization.

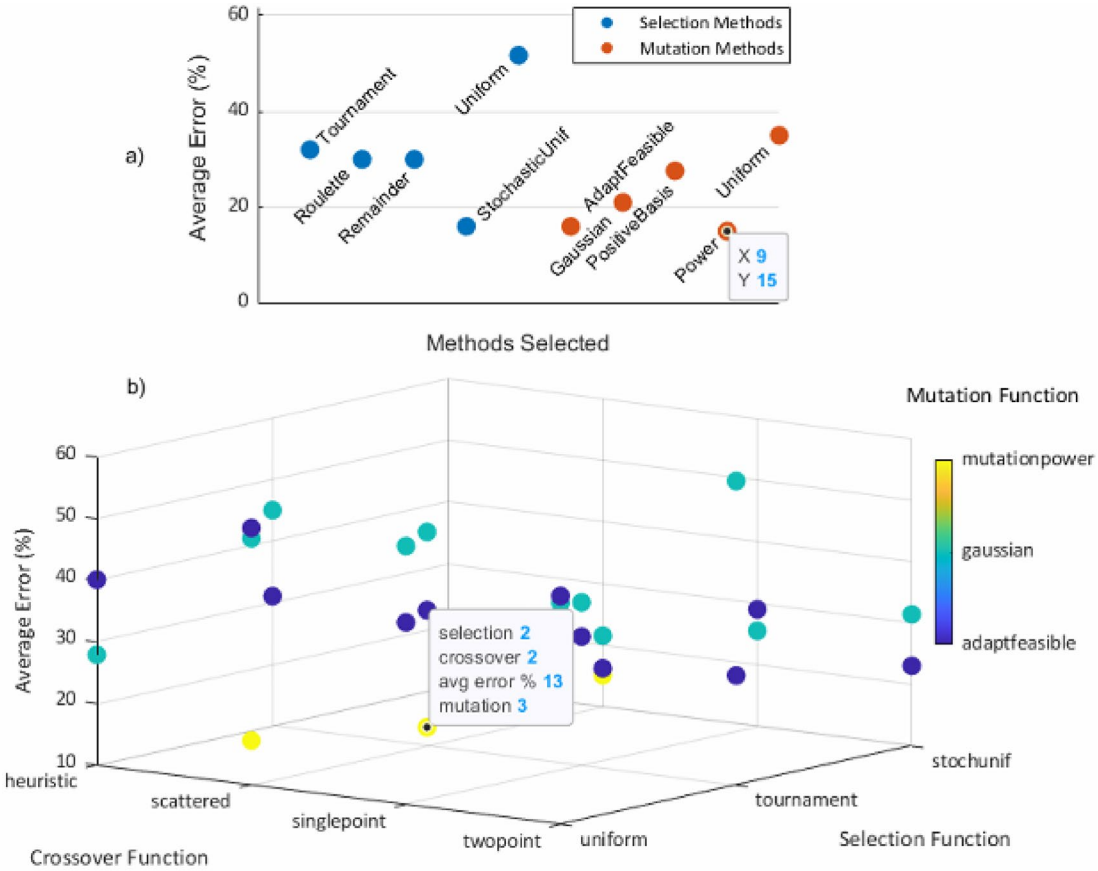

Fig. 10. Impact of Operator Methods (Selection, crossover function & Mutation) on Average Error in Optimization; **(a)** without interdependency, **(b)** with interdependency.

changes in curtailment times. PSO is the recommended option for reliable load tracking in varied grid settings due to its overall consistent and adaptable conduct as can be shown in Fig. 12.

Accurate tracking becomes harder during curtailment because the load drops abruptly to much lower, highly fluctuating values, which rapidly reshapes the cost landscape (shifted optima, steeper gradients, increased multimodality and noise). Algorithmically this penalizes methods that rely on local polling or slow generational adaptation: Pattern Search performs local polls and progressively reduces its step size, so when the optimum moves far from the current iterate it needs many iterations and step–size reductions to escape the old basin; Pareto-style multiobjective evolutionary methods preserve diversity and rebalance trade-offs across generations, so a sudden change forces them to re-establish dominance relations and rebuild the Pareto front, delaying focused exploitation. By contrast, PSO's velocity-based, population-wide updates (personal-best + global-best guidance) allow larger, directed jumps and therefore faster recovery after abrupt shifts. These differences explain why Pattern Search and Pareto methods typically underperform during curtailment.

These results illustrate their capacity to minimise a loss function that strikes a balance between two important goals: maintaining the load power at the demand level and reducing the difference between the predicted SoC and the reference one that was obtained from earlier disturbance impact study[38]. The state of charge (SoC) of the battery was successfully kept within the crucial 20%–80% range[37], guaranteeing system security and extending battery life, as shown in Fig. 13. PSO, GA, and pattern search were the tested approaches that showed the best alignment with the desired charging and discharging patterns, closely matching the dynamic response of the system.

The comparative evaluation of optimization algorithms highlights distinct strengths and limitations across the tested methods, as shown in Table 6. The Pareto-based optimizer achieved an average error of 13.7% with an average computation time of 372 s, demonstrating strong capability in handling multi-objective problems and constraints while effectively exploring trade-offs. The Pattern search algorithm, with an average error of 13.9% and computation time of 64.2 s, proved to be fast and efficient for problems with simple or well-defined constraints, making it suitable for straightforward optimization tasks. However, Pareto and Pattern adaptability are limited in curtailment or fast-changing scenarios, reducing responsiveness under dynamic conditions.

The GA recorded an average error of 8.7% and computation time of 366 s, showing good global search performance and strong handling of complex, nonlinear problem spaces. Nonetheless, its low population size may restrict search diversity, and it typically exhibits slower convergence than PSO, requiring careful parameter

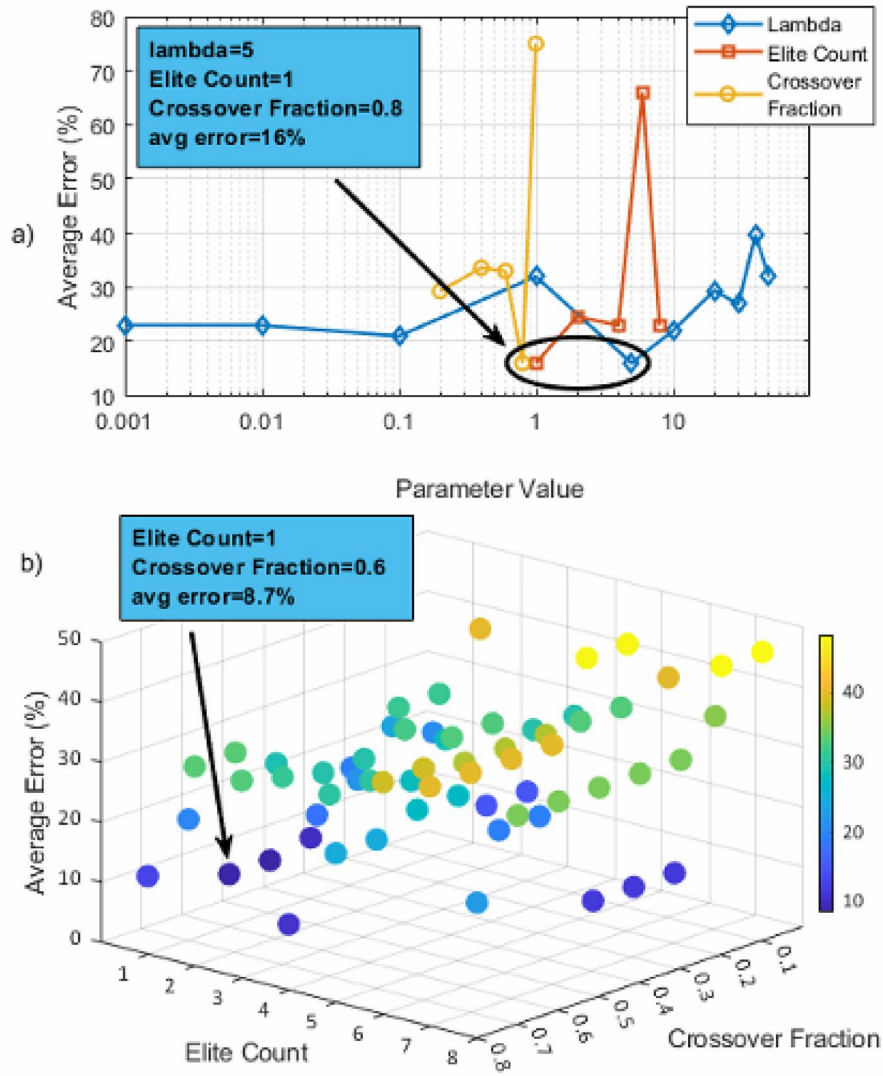

Fig. 11. Impact of GA Parameters (Elite Count, Crossover Fraction) on Average Error in Optimization; **(a)** without interdependency, **(b)** with interdependency.

adjustment for effective performance; plus, independent genetic algorithm relies on subjective user feedback which can limits its effectives.

Finally, the PSO algorithm achieved the lowest average error of 1.9% and an average computation time of 150 s, confirming its fas. convergence, robust global search ability, and effective balance between exploration and exploitation. Despite these advantages, PSO may stall or converge prematurely if not properly tuned, and a small swarm size can further reduce its exploration capability, which is acknowledged as a limitation in the study.

This study relies on offline tuning, which may not accurately reflect the dynamics of real-time MPC operation. The findings are derived from a particular microgrid setup including a battery, supercapacitor, grid, PV generation, and load. Moreover, the effectiveness of the algorithms depends on hyperparameter selection (such as population size, mutation rate, and search strategies). And also the proposed framework performs best under conditions where accurate forecasting and system information are available.

It should be noted that the execution time is reported as the average of 10 runs, since it can fluctuate due to system-related factors independent of the random seed.

Conclusion

This paper presents a practical off-line approach for tuning Model Predictive Control's cost function with constraints. These weights determine how the MPC prioritizes tracking performance, control effort, and other objectives. It assesses four optimisation algorithms for automated MPC weight adjusting based on set-point tracking: PSO, GA, Pareto Search, and Pattern Search. The approach ensures successful regulation even under numerous interacting variables and system limits by concentrating on how well the system tracks desired set points. This aligns controller behaviour with operational goals in the actual world. With a 1.9% error rate and quick convergence, PSO performed the best, making it suitable for offline applications. Although GA needed

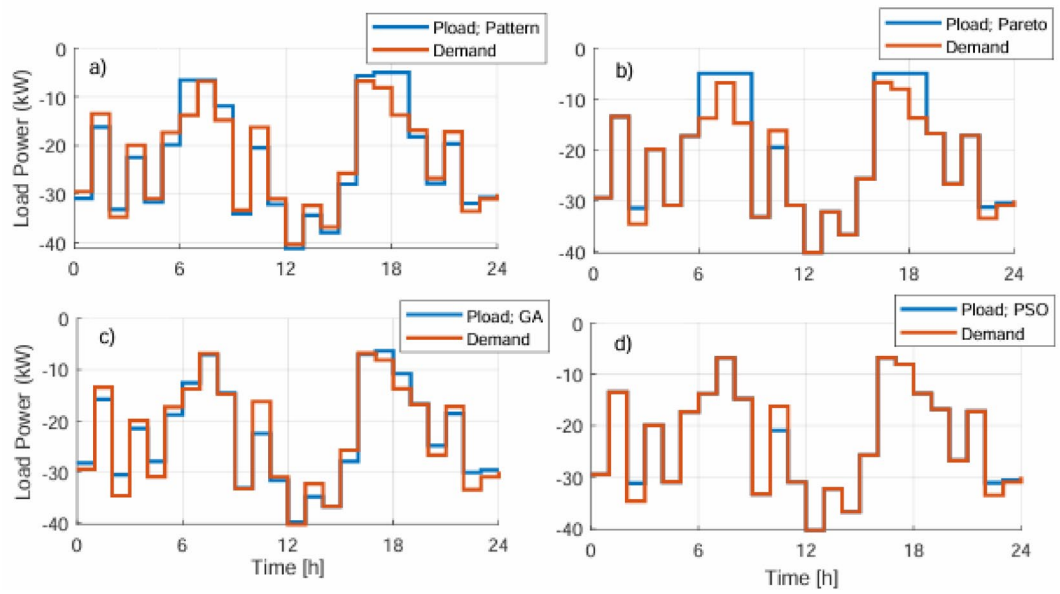

Fig. 12. Illustrates the tracking performance of load power compared to demand power for the **(a)** Pattern Search, **(b)** Pareto, **(c)** Genetic Algorithm, and **(d)** PSO methods, respectively.

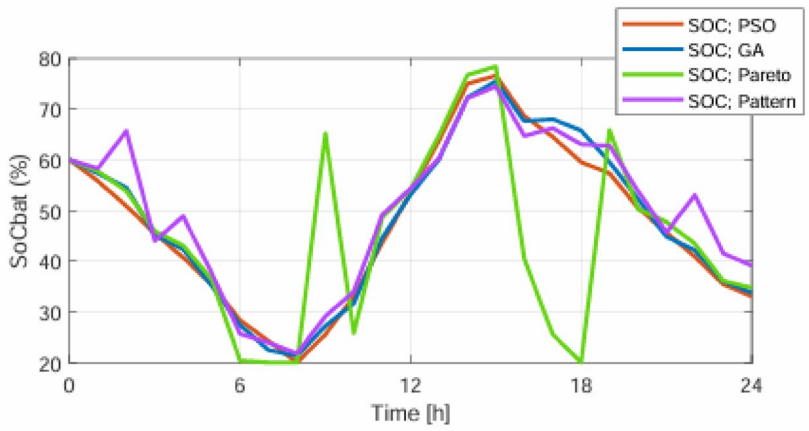

Fig. 13. Tracking Performance of Battery State of Charge Using PSO, GA, Pareto, and Pattern Search Methods.

	Average error %	Average Time(s)	Strengths	Limitations
Pareto	13.7	372	Handles multi-objective problems and constraints; explores trade-offs	Their adaptability is limited in curtailment or fast-changing scenarios.
Pattern	13.9	64.2	Fast; effective for problems with simple constraints	
GA	8.7	366	Good global search; handles complex, nonlinear spaces	Low population size may constrain search diversity; it also has a comparatively slower convergence rate than PSO and requires strict parameter adjustments.
PSO	1.9	150	Fast convergence; Balances exploration and exploitation effectively; robust global search;	It may stall or converge too soon if not properly adjusted; and small swarm size may limit exploration capability

Table 6. Comparison of optimization Algorithms.

more work and fine-tuning, it demonstrated a robust worldwide search. Pareto Search was good at handling multi-objective trade-offs, but it was less flexible and took a long time to calculate. The quickest method, Pattern Search, was vulnerable to local minima. Sensitivity analysis made clear how crucial it is to adjust parameters unique to each algorithm. Future research will concentrate on applying these optimised weights in the Microgrid model simulation.

Data availability

All data generated or analysed during this study are included in this published article.

Received: 13 August 2025; Accepted: 20 October 2025
Published online: 21 November 2025

References

1. Mayne, D. Q. et al. Constrained model predictive control: stability and optimality. *Automatica* **36** (6), 789–814 (2000).
2. Yang, X. S. *Nature-Inspired optimization algorithms* (Academic, 2020).
3. Júnior, G. A. N., Martins, M. A. & Kalid, R. A PSO-based optimal tuning strategy for constrained multivariable predictive controllers with model uncertainty. *ISA Trans.* **53** (2), 560–567 (2014).
4. Mc Namara, P., Negenborn, R. R., De Schutter, B. & Lightbody, G. Weight optimisation for iterative distributed model predictive control applied to power networks. *Eng. Appl. Artif. Intell.* **26** (1), 532–543 (2013).
5. Van der Lee, J. H., Svrcek, W. Y. & Young, B. R. A tuning algorithm for model predictive controllers based on genetic algorithms and fuzzy decision making. *ISA Trans.* **47** (1), 53–59 (2008).
6. Ramasamy, V., Sidharthan, R. K., Kannan, R. & Muralidharan, G. Optimal tuning of model predictive controller weights using genetic algorithm with interactive decision tree for industrial cement kiln process. *Processes* **7** (12), 938 (2019).
7. Yamashita, A. S., Zanin, A. C. & Odloak, D. Tuning the model predictive control of a crude distillation unit. *ISA Trans.* **60**, 178–190 (2016).
8. Yamashita, A. S., Zanin, A. C. & Odloak, D. Tuning of model predictive control with multi-objective optimization. *Braz. J. Chem. Eng.* **33** (2), 333–346 (2016).
9. Ira, A. S. et al. A machine learning approach for tuning model predictive controllers. In *2018 15th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, 2003–2008 (IEEE, 2018).
10. Gholaminejad, T., Khaki-Sedigh, A. & Bagheri, P. Adaptive tuning of model predictive control based on analytical results. In *2016 4th International Conference on Control, Instrumentation, and Automation (ICCIA),* 226–232 (IEEE, 2016).
11. Lu, Q., Kumar, R. & Zavala, V. M. MPC controller tuning using Bayesian optimization techniques. *arXiv preprint arXiv:2009.14175* (2020).
12. Stróżecki, W., Ait Oufroukh, N., Kebbati, Y., Ichalal, D. & Mammar, S. Automatic tuning of mpc for autonomous vehicle using bayesian optimization. In *2021 IEEE International Conference on Networking, Sensing and Control (ICNSC)*, Vol. 1, 1–6 (IEEE, 2021).
13. Vallerio, M., Van Impe, J. & Logist, F. Tuning of NMPC controllers via multi-objective optimisation. *Comput. Chem. Eng.* **61**, 38–50 (2014).
14. Federico, L. S. An Algorithm for Tuning NMPC Controllers with Application to Chemical Processes (2016).
15. Bagheri, P. & Khaki-Sedigh, A. An analytical tuning approach to multivariable model predictive controllers. *J. Process Control*. **24** (12), 41–54 (2014).
16. Jardine, P. T., Givigi, S. N. & Yousefi, S. Experimental results for autonomous model-predictive trajectory planning tuned with machine learning. In *2017 Annual IEEE International Systems Conference (SysCon)*, 1–7 (IEEE, 2017).
17. Yamashita, A. S., Alexandre, P. M., Zanin, A. C. & Odloak, D. Reference trajectory tuning of model predictive control. *Control Eng. Pract*. **50**, 1–11 (2016).
18. Luzi, M., Vaccarini, M. & Lemma, M. A tuning methodology of model predictive control design for energy efficient Building thermal control. *J. Building Eng.* **21**, 28–36 (2019).
19. Mehndiratta, M., Camci, E. & Kayacan, E. Automated tuning of nonlinear model predictive controller by reinforcement learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3016–3021 (IEEE, 2018).
20. Turki, M., Langlois, N. & Yassine, A. A practical tuning approach for multivariable model predictive control. In *2017 13th IEEE International Conference on Control & Automation (ICCA)*, 1107–1112 (IEEE, 2017).
21. Abrashov, S. et al. Model predictive control tuning: methods and issues. Application to steering wheel position control. *IFAC-PapersOnLine* **50** (1), 11331–11336 (2017).
22. Olesen, D. H., Huusom, J. K. & Jørgensen, J. B. A tuning procedure for ARX-based MPC of multivariate processes. In *2013 American Control Conference*, 1721–1726 (IEEE, 2013).
23. Garriga, J. L. & Soroush, M. Model predictive control tuning methods: A review. *Ind. Eng. Chem. Res.* **49** (8), 3505–3515 (2010).
24. Mohammadi, A., Asadi, H., Mohamed, S., Nelson, K. & Nahavandi, S. Multiobjective and interactive genetic algorithms for weight tuning of a model predictive control-based motion cueing algorithm. *IEEE Trans. Cybernetics*. **49** (9), 3471–3481 (2018).
25. Le, V. A. & Malikopoulos, A. A. Optimal weight adaptation of model predictive control for connected and automated vehicles in mixed traffic with Bayesian optimization. In *2023 American Control Conference (ACC)*, 1183–1188 (IEEE, 2023).
26. Santos, J. E. W., Trierweiler, J. O. & Farenzena, M. Model predictive control tuning strategy for non-square systems and range-controlled variables based on multi-scenarios approach. *Ind. Eng. Chem. Res.* **56** (40), 11496–11506 (2017).
27. Odgaard, P. F., Larsen, L. F., Wisniewski, R. & Hovgaard, T. G. On using Pareto optimality to tune a linear model predictive controller for wind turbines. *Renew. Energy*. **87**, 884–891 (2016).
28. Bemporad, A. & de la Peña, D. M. Multiobjective model predictive control. *Automatica* **45** (12), 2823–2830 (2009).
29. Lewis, R. M. & Torczon, V. Pattern search methods for linearly constrained minimization. *SIAM J. Optim.* **10** (3), 917–941 (2000).
30. Yameen, M. Z. et al. Hybrid GOA and PSO optimization for load frequency control in renewable multi source dual area power systems. *Sci. Rep.* **15**, 17549. https://doi.org/10.1038/s41598-025-01481-6 (2025).
31. Yameen, M. et al. Improving frequency stability in grid-forming inverters with adaptive model predictive control and novel COA-jDE optimized reinforcement learning. *Sci. Rep.* **15**, 16540. https://doi.org/10.1038/s41598-025-00896-5 (2025).
32. Yameen, M. Z. & Lu, Z. Enhancing microgrid stability with a PD-PIDA controlled STATCOM using a MGOA for fast-frequency response. *J. Renew. Sustain. Energy*, **17**(1) (2025).
33. Yameen, M. Z. et al. Improvement of LVRT capability of grid-connected wind-based microgrid using a hybrid GOA-PSO-tuned STATCOM for adherence to grid standards. *IET Renew. Power Gener.* **18** (15), 3218–3238 (2024).
34. Nzoundja Fapi, C. B. et al. Fuzzy Logic-Based maximum power point tracking control for photovoltaic systems: A review and experimental applications. *Arch. Computat Methods Eng.* **32**, 2405–2428. https://doi.org/10.1007/s11831-024-10210-7 (2025).
35. Li, Z., Dewantoro, G., Xiao, T. & Swain, A. A. Comparative analysis of fuzzy logic control and model predictive control in photovoltaic maximum power point tracking. *Electronics* **14**, 1009. https://doi.org/10.3390/electronics14051009 (2025).
36. Olivares, D. E. et al. Trends in microgrid control. *IEEE Trans. Smart Grid*. **5** (4), 1905–1919 (2014).
37. Nassereddine, K., Turzynski, M., Bielokha, H. & Strzelecki, R. Simulation of energy management system using model predictive control in AC/DC microgrid. *Sci. Rep.* **15** (1), 5388 (2025).
38. Nassereddine, K., Turzyński, M., Lukianov, M. & Strzelecki, R. Advancing Solar Energy: Machine Learning Approaches for Predicting Photovoltaic Power Output. In *2024 IEEE 18th International Conference on Compatibility, Power Electronics and Power Engineering (CPE-POWERENG)*, 1–6 (IEEE, 2024).
39. Open Power System Data. (n.d.). Open Power System Data – A platform for open data of the European power system. Retrieved July 9, 2025. https://open-power-system-data.org/

40. MathWorks. (n.d.). Particle swarm optimization algorithm. MATLAB & Simulink. Retrieved July 9, 2025. https://www.mathworks.com/help/gads/particle-swarm-optimization-algorithm.html
41. Lambora, A., Gupta, K. & Chopra, K. Genetic algorithm-A literature review. In *2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon),* 380–384 (IEEE, 2019).
42. MathWorks. (n.d.). Genetic algorithm. MATLAB & Simulink. Retrieved July 9, 2025. https://www.mathworks.com/help/gads/genetic-algorithm.html
43. MathWorks Global Optimization Toolbox – Genetic Algorithm, *MathWorks Help Center*. (2025). https://www.mathworks.com/help/gads/how-the-genetic-algorithm-works.html
44. MathWorks. (n.d.). Pareto search algorithm. MATLAB & Simulink. Retrieved July 9, 2025. https://www.mathworks.com/help/gads/paretosearch-algorithm.html
45. MathWorks. (n.d.). Pattern search. MATLAB & Simulink. Retrieved July 9, 2025. https://www.mathworks.com/help/gads/patternsearch.html
46. Richards, M. & Ventura, D. Choosing a starting configuration for particle swarm optimization. *Neural Networks*, 2309–2312. (2004).
47. Eberhart, R. & Kennedy, J. A new optimizer using particle swarm theory. In *MHS'95. Proceedings of the sixth international symposium on micro machine and human science*, 39–43 (IEEE, 1995).
48. Shi, Y. & Eberhart, R. A modified particle swarm optimizer. In *1998 IEEE international conference on evolutionary computation proceedings. IEEE world congress on computational intelligence (Cat. No. 98TH8360)*, 69–73 (IEEE, 1998).
49. Freire, V. A., De Arruda, L. V. R., Bordons, C. & Márquez, J. J. Optimal demand response management of a residential microgrid using model predictive control. *IEEE Access*. **8**, 228264–228276 (2020).
50. Kennedy, J., Eberhart, R. C. & Shi, Y. The particle swarm. *Swarm Intelligence*, 287–325 (2001).
51. Trelea, I. C. The particle swarm optimization algorithm: convergence analysis and parameter selection. *Inform. Process. Lett.* **85** (6), 317–325 (2003).
52. Clerc, M. Particle swarm optimization. *Part Swarm Optim*, 1–17 (2010).
53. Shi, Y. & Eberhart, R. C. Fuzzy adaptive particle swarm optimization. In *Proceedings of the 2001 congress on evolutionary computation (IEEE Cat. No. 01TH8546)*, Vol. 1, 101–106 (IEEE, 2001).
54. Boyabatli, O. & Sabuncuoglu, I. Parameter selection in genetic algorithms. *J. Systemics Cybernetics Inf.* **4** (2), 78 (2004).
55. Abu-Lebdeh, G. & Benekohal, R. F. Convergence variability and population sizing in micro-genetic algorithms. *Computer-Aided Civil Infrastructure Eng.* **14** (5), 321–334 (1999).
56. Abdoun, O. & Abouchabaka, J. A comparative study of adaptive crossover operators for genetic algorithms to resolve the traveling salesman problem. *arXiv preprint arXiv:1203.3097* (2012).

Author contributions

K.N. developed the methodology, performed simulations, data analysis, literature review and wrote the main manuscript text. M.T. supervised the research and contributed to the technical validation. All authors reviewed and approved the final manuscript.

Funding

No, this project did not receive any funding.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to K.N.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025